

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

**«Реализация межсервисного взаимодействия посредством
очередей сообщений»**

Выполнил:

Аплеев Дмитрий

K3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить RabbitMQ и сделать через него взаимодействие между сервисами.

Вариант 5. Сервис для обмена рецептами и кулинарных блогов.

Ход работы

1. Зачем очередь

Рецепты лежат в `recipe_db`, а лайки, комментарии и сохранения в `social_db`. Когда рецепт удаляют, связанные данные в `social_db` тоже нужно удалить. Раньше `recipe-service` отправлял HTTP запрос в `social-service`. Если `social-service` был выключен, запрос терялся и данные оставались навсегда.

Теперь `recipe-service` кладёт сообщение в очередь `recipe.deleted` и не ждёт ответа. `social-service` забирает сообщение и удаляет данные. Если он выключен, сообщение ждёт в очереди, пока он не включится.

```
recipe-service —> очередь recipe.deleted —> social-service  
{ "recipe_id": 5 }
```

2. Подключение

RabbitMQ запущен в Docker из образа `rabbitmq:3-management`. В `recipe-service` и `social-service` установлена библиотека `amqplib`, адрес RabbitMQ задан в `.env`.

```
npm install amqplib  
npm install -D @types/amqplib  
  
RABBITMQ_URL=amqp://guest:guest@localhost:5672
```

3. Отправка в `recipe-service`

Подключение создаётся при первом удалении рецепта и дальше используется повторно. Очередь `durable` и сообщение `persistent` сохраняются на диск, поэтому переживают перезапуск RabbitMQ.

```
export const publishRecipeDeleted = async (recipeId: number) => {  
  const channel = await getChannel();  
  const message = JSON.stringify({ recipe_id: recipeId });  
  
  channel.sendToQueue(RECIPE_DELETED_QUEUE, Buffer.from(message), {  
    persistent: true,  
  });  
};
```

Функция вызывается в контроллере после удаления рецепта.

```
await this.repository.delete({ id });

try {
  await publishRecipeDeleted(id);
} catch (error) {
  console.error('RabbitMQ is unavailable:', error);
}
```

4. Получение в social-service

После подключения к базе social-service подписывается на очередь. На каждое сообщение он удаляет комментарии, лайки и сохранения рецепта и отвечает ack. Если удалить не получилось, nack возвращает сообщение в очередь. Если RabbitMQ ещё не запущен, подключение повторяется каждые 5 секунд.

```
await channel.consume(RECIPE_DELETED_QUEUE, async (message) => {
  if (!message) {
    return;
  }

  const { recipe_id } = JSON.parse(message.content.toString());

  try {
    await deleteRecipeData(recipe_id);
    channel.ack(message);
  } catch (error) {
    channel.nack(message, false, true);
  }
});
```

HTTP эндпоинт DELETE /v1/internal/recipes/{id} из ЛР2 больше не нужен и удалён.

5. Проверка

- social-service остановлен, рецепт удалён через BFF, ответ 204
- в панели RabbitMQ в очереди recipe.deleted лежит одно сообщение
- social-service запущен, сообщение сразу обработано, очередь пустая
- лайки и комментарии удалённого рецепта из social_db удалились

```
Listening to RabbitMQ queue "recipe.deleted"
recipe 5 deleted: comments, likes and saves removed
```

Вывод

Удаление связанных данных между сервисами переведено с HTTP на очередь RabbitMQ. Сообщения не теряются, даже если social-service в момент удаления не работает.